
SKALA 4기 과제 제출 보고서 · 설계 산출물

KV cache 최적화 기술 다관점 평가 Agentic RAG 설계서

- 판교 9반 1조 -

SKALA

과정 | SKALA 4기 · AI

반/조 | 9반 1조

팀원 | P284 김정인

팀원 | P285 김지수

팀원 | P286 김진수

팀원 | P304 전진만

팀원 | P307 정원준

제출일 2026. 09. 22.

문서 정보

과제명	KV cache 최적화 기술 다관점 평가 Agentic RAG 설계서	과제 유형	팀
과정	SKALA 4기	반 / 조	9반 1조
작성자	김정인, 김지수, 김진수, 전진만, 정원준	제출 일	2026. 09. 22.
담당 강사	배기주 교수님	문서 버전	v1.3 (설계 개정)

요약

- **목적:** 데이터센터 장문맥 LLM 서버의 KV cache 병목을 푸는 SW 기술(Google TurboQuant)과 HW 기술(SK hynix ITME)이 TRL·시장·이해관계자·도메인 관점에서 **어떻게 다르게 인식되는지** 비교하는 Agentic RAG를 설계한다. 기술 간 우열은 판정하지 않는다.
- **선정:** 조가 직접 선정(2안)했고, 에이전트는 검증 결과만 기록한다(그래프 안에 재선정 분기 없음). 후보 6개를 가중 기준으로 채점한 결과 TurboQuant 4.35, ITME 4.50이었다. 두 기술은 **같은 KV cache 용량 병목을, 같은 서버 시점에서, 서로 다른 시스템 계층으로** 해결하므로 공정하게 비교할 수 있다.
- **RAG:** 논문 6편(136p)을 잘 인식 청크 159개로 인덱싱했다. 임베딩은 한국어→영어 42문항 자체 평가로 골랐다. 사전 적합성(다국어·모델 한계 900토큰 이상·오픈소스)을 통과한 후보 중 bge-m3를 선정했다(MRR 0.703, min(SW, HW) 0.659). 최종 검색 구성(3중 RRF + reranker)은 구현 조건(max_seq_length 2,048)으로 다시 측정해 Hit@5 0.976, MRR 0.863이다. 모두 구성 선택에 쓴 42문항 기준의 **개발 지표**이다.
- **그래프:** 에이전트 7개(Notion 6개 + Judge), State 키 27개로 구성한다. 공통 검색·관점 내부·관점 재실행·보고서 검수의 4개 루프에 모두 hit 수 한도가 있다. Judge가 미달로 판정한 관점만 **retry_router**가 **Send**로 다시 실행한다.
- **편향 방지:** 찬반 양면 검색(기술 고유 근거만 할당량으로 인정), 원 출처 계열 기준 50% 상한, 인용 필수, 우열 어휘 검사, 상위 모델 Judge를 적용한다.
- **한계:** 평가셋의 개발/테스트 미분리(42문항, 1문항 = 2.4%p), 같은 계열 Judge, ITME 공개 후 약 3개월, 평가 주체(SK 교육과정)의 소속 편향 가능성.

A. 대상 기술 선정 (선정 사유 포함)

A.1 분석 배경: KV cache가 왜 병목인가

LLM은 토큰을 하나씩 생성하면서, 앞에서 계산한 Key-Value 텐서를 KV cache에 저장해 두고 다시 사용한다. 재계산 비용은 사라지지만, KV cache의 크기는 **문맥 길이와 동시 요청 수에 비례해 선형으로** 커진다. 그래서 KV cache는 "연산 병목"을 "메모리(HBM) 병목"으로 바꿔 놓는다.

KV cache 크기 = $2(K \cdot V) \times \text{레이어 수} \times \text{KV 헤드 수} \times \text{헤드 차원} \times \text{토큰 수} \times \text{원소 바이트}$. 예: Llama-3.1-8B(32 레이어, KV 헤드 8, 헤드 차원 128, FP16)는 토큰당 128 KiB이다. 128K 토큰 요청 1건이면 16 GiB로, 모델 가중치(약 16 GB)와 비슷한 크기이다. 80 GB GPU 1장에서 가중치를 빼면 64 GB가 남는다. $64 \text{ GB} \div 17.2 \text{ GB}(16 \text{ GiB}) \approx 3.7$ 이므로 128K 문맥 세션을 **온전히 3개만** 유지할 수 있고, 활성값 메모리까지 고려하면 더 적다.

이 병목을 푸는 접근은 두 진영으로 나뉜다. **SW 진영은 데이터를 작게** 만들고(양자화·압축, 어텐션 구조 변경), **HW 진영은 담을 공간을 넓힌다**(HBM 밖의 호스트-CXL-스토리지 계층으로 확장). 둘은 대립하는 것처럼 보이지만 실제로는 함께 쓰이는 경우가 많다. 예를 들어 압축된 KV는 하위 메모리 계층으로 옮기기도 쉽다.

A.2 선정 기준

후보 6개(Doc Pool)를 네 가지 기준으로 5점 척도 채점하고, 가중합 $\Sigma(\text{기준 점수} \times \text{가중치})$ 을 계산한다. 가중합과 별개로 **제외 조건** 중 하나라도 해당하면 후보에서 뺀다.

기준	가중치	판단 질문
비교 공정성	35%	두 기술 모두 모델 학습 이후 동일한 적용 시점(추론·서빙 단계) 에서 KV cache 용량 병목을 직접 다루는가?
근거 확보성	25%	원 논문과 독립적인 보조 자료를 확보할 수 있고, 장점과 한계를 함께 검증할 수 있는가?
산업 연관성	25%	GPU 서빙 생태계나 CXL 메모리 생태계와 연결되어, 실제 도입 주체와 이해관계자를 식별할 수 있는가?
최신성	15%	장문맥·에이전트 워크로드의 최근 문제와 연결되는가?

제외 조건: ① KV cache를 직접 평가 대상으로 삼지 않음 ② 원문(1차 자료)을 확보할 수 없음 ③ 비교 대상과 적용 시점이 크게 다름(예: 사전학습 단계) ④ 성능 수치의 실험 환경과 기준선이 공개되지 않음

A.3 후보 6개 평가표 (설계 시점 팀 채점)

후보	진영	공정성	근거	산업	최신	가중합	채점 근거 요약
TurboQuant	SW	5	3	5	4	4.35	재학습 없이 서빙 중 KV를 온라인 양자화. 설계 시점에 확보한 자료는 Google Research·DeepMind 논문(2025-04)과 Google 공식 블로그(2026-03-24)로 모두 Google 자료 이다. ITME와 같은 기준으로 근거 점수 3(v1.1 수정, 4→3)
KIVI	SW	5	4	3	2	3.80	서빙 단계 2bit 양자화, ICML 2024. TurboQuant 논문의 직접 비교 기준선이라 baseline 역할 로 활용

후보	진영	공정성	근거	산업	최신	가중합	채점 근거 요약
DeepSeek MLA	SW	1	5	4	2	2.90	어텐션 구조를 바꿔 사전학습이 필요 → 제외 조건 ③ 해당. 대조점으로만 활용
ITME	HW	5	3	5	5	4.50	vLLM(v0.17.0) 위에 구현, 모델 변경 없음. SK hynix 양산급 CMM과 FPGA로 실측(2026-06). 1차 자료가 벤더 단일 출처 라 근거 점수는 3
InfiniGen	HW	4	3	2	2	2.95	호스트 메모리 오프로딩과 선택적 프리패치(OSDI 2024). 오프라인 가중치 변환(skewing)이 필요해 공정성 4점
CXL-PNM	HW	3	2	3	4	2.90	CXL 메모리 안에 PNM 연산기를 두어 실행 위치 자체를 바꿈 (새 연산 HW 필요). 시뮬레이션 중심 preprint(2025-10)

진영별 최고점: SW는 TurboQuant(4.35), HW는 ITME(4.50). MLA는 제외 조건 ③으로 빠지고, KIVI-InfiniGen-CXL-PNM은 보고서에서 기준선·대안 기술로 인용한다.

A.4 선정 결과와 사유

공정 비교의 근거는 두 기술이 같은 종류라서가 아니다. **같은 KV cache 용량 병목을, 같은 서빙 시점에서, 서로 다른 시스템 계층으로 해결하기 때문이다**. TurboQuant는 데이터 표현 계층(비트 수)에서, ITME는 메모리 계층(저장 위치)에서 해결한다.

사유	SW · Google TurboQuant	HW · ITME (SK hynix, CXL-hybrid 계층 메모리)
대칭 평가 가능성	재학습 없이 서빙 단계에서 KV를 양자화 → 모델을 바꾸지 않는 기술끼리 같은 기준·질문 템플릿으로 비교 가능	vLLM 위에 구현된 서빙 계층 기술이고 모델을 바꾸지 않음 → TurboQuant와 같은 평가를 적용 가능
접근 방식 대비	데이터를 작게: 3.5bit/채널에서 품질 중립, 2.5bit에서 소폭 저하 보고(LongBench-NIAH, Llama-3.1-8B-Ministral-7B)	공간을 넓게: KV를 양자화하지 않고 CXL-hybrid 메모리 (T3.5)로 TB 규모 확장. NVMe-oF 대비 처리량 1.80배, CPU 오프로드 대비 최대 35.7% 향상 보고
관점 간 인식 분기	"추가 HW 없이 기존 GPU에서 메모리 절감"이라는 기대와 "정확도 영향·서빙 엔진 통합 수준"이라는 우려가 공존	"무손실 용량 확장"이라는 기대와 "원격 계층 지연·인프라 비용·배포 성숙도"라는 제약이 공존
이해관계자 다양성	서빙 엔진 개발자, 클라우드 사업자, 경쟁 KV 압축 진영 (KIVI, NVIDIA KVTC 등), 메모리 업계	메모리 벤더, 서버·클라우드 사업자, CXL Consortium, 원격 플래시·NVMe 기반 대안 진영(NVIDIA CMX 등). TurboQuant와 이해관계자 집단이 거의 겹치지 않음
TRL 관찰 가치	알고리즘 논문과 오픈소스 구현 중심 → 연구 성과가 프로덕션 도입으로 이어지는 단계를 관찰	양산급 모듈로 실측했지만 상용 배포 정보는 제한적 → TRL 4~6 구간의 공개 정보 격차를 관찰
최신성·산업 연관성	빅테크 공개(블로그 2026-03)로 시장·미디어 반응 자료가 풍부	2026-06 공개, 국내 메모리 반도체(CXL) 생태계와 직접 연결
환경·결합 검토	온디바이스 등 메모리 제약 환경에서 평가가 달라지는지 확인 가능(H3)	TurboQuant와 결합 가능한 보완 관계인지 관점별로 검토 가능(H4)

A.5 핵심 contribution 분석 (원 논문 페이지 근거)

각 행의 근거는 원 논문의 해당 페이지에서 확인했다(p.는 arXiv PDF 기준). 압축률·속도·처리량 수치는 contribution을 뒷받침하는 실험 결과로만 표 아래에 따로 적는다.

TurboQuant (arXiv 2504.19874)

기존 방식의 구조적 문제	논문의 핵심 contribution	개선되는 부분	새로 생기거나 남는 제약	본 프로젝트 평가 항목
기존 벡터 양자화는 최적 왜곡률에 미치지 못한다(p.1).	TurboQuant_mse : 무작위 회전으로 각 좌표가 Beta 분포를 따르게 하고, 좌표별 최적 scalar quantizer를 적용한다(p.1-5, Alg.1 p.10).	정보이론 하한과 상수배(≈ 2.7) 차이의 왜곡률(p.1)	회전·양자화 연산이 서빙 경로에 추가된다. 서빙 엔진 통합과 처리량은 논문 범위 밖이다.	도메인(통합 난이도·지연), TRL(프레임워크 반영)
MSE 최적 양자화기는 내적 추정에 편향 을 만든다(p.1). 어텐션 점수는 내적이다.	TurboQuant_prod : MSE 양자화 후 잔차에 1-bit QJL을 적용해 불편(unbiased) 내적 양자화 를 만든다(p.1, Alg.2 p.12).	어텐션 logit 추정 편향 보정	잔차 부호 비트와 노름을 추가로 저장해야 한다. MSE 변형과 내적 변형 중 하나를 골라야 한다.	도메인(정확도 영향)
데이터 의존(오프라인) 양자화는 스트리밍 KV에 쓰기 어렵다. KIVI 등은 생성 토큰을 양자화하지 않는다(p.18).	data-oblivious-online 알고리즘으로, 생성 중인 토큰에도 양자화를 적용한다(p.1-18).	캘리브레이션 없이 장문 생성에도 적용 가능	2.5bit에서 소폭 품질 저하(p.1). 비정수 비트는 이상치 채널 분리의 결과이다(p.18). 실험은 단일 A100-8GB 모델이다(p.15).	도메인(정확도), 시장(발표 주장과 실험 조건 대조)

뒷받침 실험 결과: 3.5bit/채널에서 품질 중립, 2.5bit에서 소폭 저하(LongBench-NIAH, p.1). Google 블로그는 KV 메모리 6배 이상 절감과 "최대 8배" 속도를 보고한다. "최대 8배"는 H100에서 4bit 대 **32bit 키**의 attention logit 계산 속도를 비교한 값이므로 기준선 해석에 주의해야 한다.

ITME (arXiv 2606.12556)

기존 방식의 구조적 문제	논문의 핵심 contribution	개선되는 부분	새로 생기거나 남는 제약	본 프로젝트 평가 항목
DPU 기반 JBof로 NVMe-of 처리를 오프로딩해도 SW 최적화 부담과 비용 효율 문제가 남는다(p.1). 호스트 메모리는 CPU 소켓·채널 수에 묶인다(p.1).	CXL-hybrid 메모리 (NAND + HW 관리 DRAM 캐시, CPU-less NUMA 노드)로 TB급 byte-addressable 원격 메모리를 제공한다(p.1-4). 저가 RNIC를 사용해 DPU 의존을 없앤다(p.3).	직접 바이트 주소 접근으로 SW 스택이 단순해진다. 호스트 한계를 넘는 KV 용량을 확보한다(p.1).	원격 CXL-hybrid 메모리의 지연(로컬 GPU 메모리 대비, p.9), NAND 수명 관리 필요(p.5), 전용 CXL 장치·RNIC 인프라	도메인(지연·비용·통합), 시장(CXL 생태계)
필요할 때 원격에서 가져오면 전송 지연이 그대로 드러난다.	모델 가중치와 prefix KV의 결정적(예측 가능한) 접근 패턴 을 이용한 파이프라인 다계층 DMA 선제 이동(prefetch) . CXL 장치에서 GPU로 직접 전송한다(p.1-3).	전송 지연 은닉, 재계산 대비 이득(p.9)	working KV처럼 예측성이 낮은 데이터는 대상이 아니다. KV 미스는 GPU 재계산으로 처리한다(p.7).	도메인(W2 다중 턴 적합 조건)

기존 방식의 구조적 문제	논문의 핵심 contribution	개선되는 부분	새로 생기거나 남는 제약	본 프로젝트 평가 항목
- (검증 방식)	양산급 SK hynix CMM과 PCIe Gen5 NVMe로 성능 잠재력을 평가하고, FPGA 프로토타입 으로 기능을 검증한다. vLLM v0.17.0 위에 구현했다(p.1-9).	시뮬레이션이 아닌 실제 HW 근거	벤더 저자·벤더 제품 기반이다. 워크로드는 ShareGPT 다중 턴, Llama-3.1 8B/70B이다(p.9).	TRL(4~6 구간 판단), 이해관계자(벤더 라벨)

뒷받침 실험 결과: NVMe-oF 기반 분리형 스토리지 대비 처리량 1.80배(p.2), CPU 오프로드 대비 최대 35.7% 향상(p.1).

A.6 알려진 약점 (선정 검증 단계에서 재확인)

- **ITME**: 저자 전원이 SK hynix 소속이고 제품(CMM) 기반으로 실측했다. 1차 근거가 벤더 자료이므로, 시장·이해관계자 관점에서는 **제3자 출처를 따로 확보해야** 한다(출처 다양성 규칙 적용).
- **TurboQuant**: 논문 실험은 단일 A100에서 품질과 왜곡률을 중심으로 이루어졌고, 서빙 엔진 통합과 처리량 수치는 논문에 없다. 블로그의 "최대 8배"는 **H100에서 4bit 대 32bit 키의 attention logit 계산 속도를** 비교한 값이라 기준선 해석에 주의해야 한다.
- **평가 주체의 소속 편향 가능성**: 평가 주체가 SK 교육과정 소속이고 ITME는 SK hynix 기술이다. 선정 기준표와 모든 관점에 대칭 기준을 적용하고, 이 가능성을 보고서 한계점에 명시한다.
- 선정은 사람이 했으므로, 기술 조사 Agent의 `selection_validator`가 원문을 근거로 세 가지를 다시 검증한다. ① 같은 문제를 다루는가 ② 적용 시점이 같은가 ③ 공개 근거가 충분한가. 하나라도 미충족이면 사유와 대체 후보를 `selection_validation`과 `warnings`에 기록하고 실행은 계속한다. 2안(Human 선정)이므로 그래프 안에서 기술을 바꾸지 않는다. 선정을 바꿀지는 팀이 실행 밖에서 결정하고 `--tech`로 다시 실행한다. 검증 결과(통과 여부·약점·대체 후보)는 보고서 2.4절에 그대로 신는다.

B. 설계: 기술 선정 방식 · RAG 적용 대상 · Embedding 모델

B.1 기술 선정 방식: Human 기반(2안)과 에이전트 사후 검증

팀이 A의 기준으로 TurboQuant와 ITME를 선정하고(`config.yaml`의 `selected_techs`, CLI `--tech sw=turboquant,hw=itme`로 교체 가능), 기술 조사 Agent가 선정 타당성을 사후 검증한다. 에이전트가 검색 결과만으로 대상을 정하면 **최신 자료에 치우치거나 비교 계층이 어긋날** 위험이 있다. 이 구조는 그 위험을 줄이면서, 사람의 선택을 그대로 확정하지 않고 근거로 다시 확인하기 위한 것이다. 에이전트는 **다시 고르지 않고 검증만 한다**. 그래서 그래프에는 1안(에이전트 선정)이나 재선정 분기를 두지 않는다.

B.2 RAG 적용 대상

에이전트	RAG	검색 대상	산출물
기술 조사 (검증·개요·TRL)	0	선정 기술 원 논문(primary)과 기준선·대안 논문	작동 원리, 실험 조건, 한계, TRL 근거

이해관계자 관점은 **최신 반응**이 핵심이라 논문 RAG의 이점이 작아 웹만 쓴다(Notion 가이드와 일치). 평가 종합, Judge, 보고서 에이전트는 새 사실을 검색하지 않는다. 그래서 근거 없는 내용이 끼어들 수 있는 경로가 관점 에이전트로만 한정된다.

공통 검색 경로(query_planner → hybrid_retriever → retrieval_grader)의 결과는 tech_research만 사용한다. 따라서 query_planner의 질의 범위는 **기술 개요와 공통 근거용**으로 한정한다. 관점별 찬반 질의는 각 관점 에이전트가 내부 Loop에서 직접 만든다.

B.3 코퍼스 구성 (200페이지 한도)

논문	arXiv	페이지	진영	역할	청크	활용
TurboQuant	2504.19874	25	SW	primary	21	선정 기술 1차 근거
DeepSeek-V2 (MLA)	2405.04434	52	SW	alternative	44	구조 변경형 대안(대조)
KIVI	2402.02750	15	SW	baseline	21	양자화 계열 기준선
InfiniGen	2406.19707	18	HW	alternative	26	호스트 오프로딩 대안
ITME	2606.12556	13	HW	primary	24	선정 기술 1차 근거
CXL-PNM	2511.00321	13	HW	alternative	23	CXL+PNM 대안
합계		136			159	한도 200p의 68% (scripts/download_papers.py가 페이지 수 자동 검증)

선정 기술 논문 2편만 넣으면 "기준선 대비 어떤 위치인가"라는 질문에 답할 근거가 없다. 그래서 Doc Pool 6편을 **모두** 넣고 **role** 메타데이터로 1차 근거와 비교 근거를 구분한다. 에이전트는 **tech-role** 필터로 필요한 범위만 검색한다.

B.4 문서 처리·인덱싱 파이프라인 (구현 완료)

- 로딩(PyMuPDF):** 글꼴 크기와 굵기로 절 제목을 인식해 청크마다 **section**을 붙인다. 40% 이상의 페이지에서 반복되는 머리글·바닥글, 쪽번호, arXiv 워터마크를 제거한다. **참고문헌 목록은 제외**하고 부록은 유지한다.
- 청킹:** 절 경계를 우선 보존하면서 900 토큰(d100k) 단위로 나누고, 15%(135 토큰)를 겹친다. 문단 → 문장 → 토큰 순으로 분할해 표나 수식 덩어리가 한도를 넘지 않게 한다. 결과는 159개 청크이다. 900은 d100k 기준 토큰 수이고, 임베딩 모델 토큰라이저로 세면 평균 약 750, 최대 1,529 토큰이다(outputs/eval/truncation_check.csv). 그래서 구현 단계에서는 bge-m3의 **max_seq_length**를 2,048로 둔다(모델 한계 8,192 이내).
- 메타데이터:** **chunk_id**, **doc_id**(arXiv), **tech**, **camp**(SW/HW), **role**(primary/baseline/alternative), **title**, **section**, **page-page_end**, **published_at**, **source_type**를 저장한다.
- 인덱스:** 청크 앞에 **제목 | 절** 머리말을 붙여 임베딩하고 FAISS(IndexFlatIP, 정규화 코사인)에 넣는다. BM25 인덱스를 함께 만든다. 코퍼스 지문(hash)이 같으면 **기존 인덱스를 재사용**하고, 없으면 자동으로 만든다.

B.5 검색 전략 (Agentic RAG)

- 질의 계획:** 공통 경로의 `query_planner`는 기술 × 개요 항목(원리·실험 조건·한계) 질의를 만든다. 관점 에이전트는 내부 Loop에서 기술 × 관점 × 입장(지지/반대) × 확인할 지표 질의를 만들고, 반대 근거는 별도 질의로 찾는다.
예: "TurboQuant 장문맥 정확도 저하를 지적하는 결과"
- 질의 보강:** 한국어 질의를 영어 검색어로 다시 쓴다. 이때 **질문만 보고** 재작성하므로 정답 정보가 섞여 들지 않는다.
- 하이브리드 검색:** 한국어 dense, 영어 dense, 영어 BM25 세 가지 순위를 **RRF(k=60)**로 합친다. 그다음 cross-encoder(bge-reranker-v2-m3)로 상위 20개를 다시 정렬하고 top-5를 전달한다.
- 품질 판정:** 공통 경로의 `retrieval_grader`는 검색 결과에 기술명, 원리, 수치 또는 한계 조건이 들어 있는지 판정한다(찬반 균형은 관점 에이전트 내부 Loop에서 판정). 부족하면 `query_rewriter`가 누락 요소를 반영해 질의를 다시 쓰고 재검색하며, **최대 2회**까지 반복한다. 그래도 부족하면 **근거 부족** 경고를 남긴다.
- 근거 사용:** 최종 주장마다 근거 ID를 1개 이상 연결한다. 홍보성 주장이나 시장 전망은 독립 출처를 함께 찾는다. 출처 50% 규칙은 C.2의 정의(분모·도메인 정규화·벤더 출처균·원 출처 계열)를 따른다.

B.6 Embedding 모델 선정: 자체 교차언어 평가

평가 목적: 리더보드 순위가 아니라, 본 과제의 실제 검색 조건인 **한국어 질의로 영어 논문 청크를 찾는 교차언어** 검색에서의 성능과 로컬 운영 비용을 직접 측정한다.

후보(오픈소스 4종): BAAI/bge-m3, intfloat/multilingual-e5-large, Qwen/Qwen3-Embedding-0.6B, sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2(경량 기준선). 모두 다국어 모델이다. 영어 전용 MiniLM-L6은 한국어 질의를 처리할 수 없어 다국어 L12로 바꿨다.

선정 절차 (3단계): ① 사전 적합성(필수 조건)으로 후보를 거르고, ② 과제 전용 실험으로 정확도를 측정하고, ③ 운영 판단으로 진영 간 균형과 비용을 확인한다. v1.0의 가중 기준표(40/20/15/15/10)는 필수 조건과 한 절 안에서 섞여 선정 방식이 두 개로 읽혔기 때문에 삭제했다.

① **사전 적합성 (필수 조건):** 교차언어(다국어) 검색 지원, **모델 고유 최대 입력 900토큰 이상**(청크 무손실 가능), 오픈소스 라이선스, 로컬(Apple Silicon) 실행 가능. 탈락한 후보도 비교를 위한 참고 기준선으로 측정 결과를 유지한다.

모델	다국어	모델 고유 최대 입력(모델 카드·config)	평가 설정 길이	평가 시 잘린 청크	라이선스	로컬 실행	① 판정
bge-m3	0	8,192	1,024	17/159	MIT	0	통과
qwen3-embedding-0.6b	0	32,768	1,024	4/159	Apache-2.0	0	통과
multilingual-e5-large	0	512	512	136/159	MIT	0	탈락 (참고 기준선)
multilingual-minilm-l12	0	128	128	159/159	Apache-2.0	0	탈락 (참고 기준선)

측정 조건 공개: 청크 900토큰은 cl100k 기준이라, 모델 토큰나이저로 세면 최대 1,529토큰이다. 평가 때 bge-m3와 Qwen3의 `max_seq_length`를 1,024로 두어 **bge-m3 17개, Qwen3 4개** 청크가 뒷부분이 잘린 채 측정되었다. 아래 수치는 이 조건의 측정값이며 바꾸지 않았다. 구현 단계에서는 `max_seq_length`를 2,048로 올린다.

평가셋 구성 절차 (eval/):

- 1. 생성(60문항):** 논문마다 절을 고르게 나눠 10개씩 청크를 뽑는다. Judge 등급 LLM(gpt-4.1)이 청크 하나에서만 확인할 수 있는 **단일·구체 사실**을 묻는 한국어 질문을 만든다. 영어 구절은 5단어 이상 복사하지 못하게 한다.
- 2. 자동 QC(57문항):** 답할 수 있는가, 구체적인가, 질문만으로 이해되는가를 LLM이 점검하고, 구절 복사 검사를 더해 57/60이 통과했다.
- 3. 복수 정답 풀링(50문항):** 57문항마다 후보 4개 모델과 BM25의 top-10 합집합을 만들었다(총 1,103개 (질문, 청크) 쌍). 이를 기준 답에 비추어 엄격하게 판정했고, 정답 청크가 5개 이상인 **포괄적 질문 7개를 제외**했다.
- 4. 균형 선택(42문항):** 50문항에서 논문별 7문항을 골라 **42문항**을 만들었다(SW 21 / HW 21). 42문항 중 **29문항**이 복수 정답이다.
- 5. 사람 검수(11문항):** 42문항 중 **11문항(26%)**을 직접 검수했다. 질문과 정답 청크 쌍, 그리고 해당 문항의 추가 정답 청크를 원문과 대조했다. LLM 판정 오류 1건(turboquant-012)을 찾아 수정했고, 내역은 `eval/qa_review.md`에 있다.
- 6. 누설 차단:** 영어 재작성 질의는 **질문만 보고** 생성한다. 초기 버전에서 정답 청크를 본 LLM이 재작성해 BM25(EN) 점수가 0.816으로 부풀려졌던 것을 발견해 바로잡았다(수정 후 0.770).

공정성 통제: 청크, 질의, FAISS 설정을 동일하게 두고 모델만 교체했다. 각 모델은 모델 카드가 권장하는 질의 설정을 썼다(e5: `query:/passage:` 접두어, Qwen3: 기본 `query` 프롬프트). Qwen3는 프롬프트 민감도가 커서(MRR 0.548→0.631) **가장 좋은 설정**으로 비교했다.

수치의 성격: 아래의 모든 Hit@K와 MRR은 모델·프롬프트·검색 구성을 고르는 데 쓴 **같은 42문항(구성 선택용 셋)**으로 낸 개발 지표 (**development metric**)이다. 독립 테스트셋 성능이 아니다.

② 과제 전용 실험. 결과 1: 임베딩 후보 (dense only, 한국어 질의, v1 측정: `max\_seq\_length` 1,024)

모델	Hit@1	Hit@3	Hit@5	MRR@10	MRR SW / HW	인덱싱(s)	질의 (ms)	MPS 메모리 (MB)
bge-m3	0.548	0.857	0.905	0.703	0.748 / 0.659	31.2	7.2	3049
qwen3-embedding-0.6b	0.500	0.738	0.810	0.631	0.770 / 0.493	27.7	10.1	3265
multilingual-e5-large	0.476	0.738	0.857	0.626	0.662 / 0.589	17.4	7.7	3081
multilingual-minilm-l12	0.262	0.405	0.500	0.377	0.378 / 0.375	0.6	1.0	1065

측정 환경: Apple M5(16 GB, MPS) · 159 청크 · 42문항 · `max_seq_length`는 ① 표의 평가 설정. 인덱싱은 전체 청크 인코딩 시간, 질의는 1건당 평균. Hit@k는 복수 정답 중 하나라도 top-k 안에 있으면 적중, MRR은 첫 적중 순위의 역수 평균(top-10).

결과 2: 검색 구성 (bge-m3 기준, 같은 평가셋, v1 측정)

검색 구성	Hit@1	Hit@3	Hit@5	MRR@10
Dense (한국어 질의)	0.548	0.857	0.905	0.703
BM25 (한국어 질의)	0.548	0.810	0.881	0.697
BM25 (영어 재작성 질의)	0.643	0.905	0.929	0.770

검색 구성	Hit@1	Hit@3	Hit@5	MRR@10
2중 하이브리드 RRF: Dense(KO) + BM25(EN)	0.643	0.929	0.929	0.786
3중 하이브리드 RRF: Dense(KO) + Dense(EN) + BM25(EN)	0.667	0.881	0.952	0.790
2중 하이브리드 RRF + reranker (참고)	0.786	0.952	0.976	0.863
3중 하이브리드 RRF + reranker ← 채택(v1에서 고정)	0.786	0.952	0.976	0.863

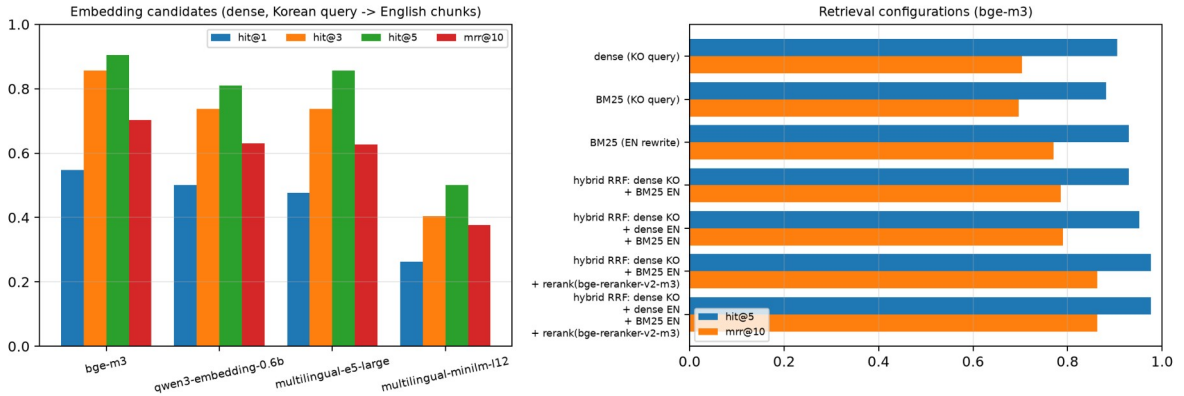


그림 1. 임베딩 후보(좌)와 검색 구성(우) 평가 결과 · outputs/eval/embedding_eval.png

③ **운영 판단:** 진영 간 균형은 $\min(\text{SW MRR}, \text{HW MRR})$ 로 정의한다. 두 진영 중 더 약한 쪽의 검색 품질이 보고서 전체의 근거 품질을 제한하기 때문이다. 이 정의는 **v1.1에서 명시한 것이고, 측정 이후에 정했다**는 점을 한계로 밝힌다. 다만 이 기준을 빼도 ②의 전체 지표 순위는 같다. 여기에 메모리, 지연, 인덱스 크기를 더해 본다.

모델	① 결과	$\min(\text{SW}, \text{HW})$ MRR	SW / HW MRR	인덱싱(s)	질의(ms)	MPS 메모리 (MB)	인덱스 (MB)
bge-m3	통과	0.659	0.748 / 0.659	31.2	7.2	3049	0.62
qwen3-embedding-0.6b	통과	0.493	0.770 / 0.493	27.7	10.1	3265	0.62
multilingual-e5-large	탈락	0.589	0.662 / 0.589	17.4	7.7	3081	0.62
multilingual-minilm-l12	탈락	0.375	0.378 / 0.375	0.6	1.0	1065	0.23

결정: 임베딩 'BAAI/bge-m3', 검색 구성 "3중 하이브리드(RRF) + bge-reranker-v2-m3". 과제 조건은 두 진영 문서를 같은 비중으로 검색하고, 900토큰 청크를 잘림 없이 인코딩할 수 있으며, 오픈소스로 로컬에서 실행하는 것이다. ① 사전 적합성에서 이 조건을 만족한 후보는 bge-m3와 Qwen3-Embedding-0.6b뿐이었다. ③에서 진영 간 균형 $\min(\text{SW}, \text{HW})$ 은 bge-m3 0.659, Qwen3 0.493이었다. Qwen3는 SW 질문 MRR이 조금 더 높았지만(0.770 vs 0.748) HW 질문에서 0.493으로 크게 떨어졌다. ②의 확인 결과로, bge-m3는 전체 Hit@1:3-5와 MRR이 모두 가장 높았다. 구현 조건(2,048)으로 다시 측정해도 순위와 채택 구성의 수치는 같았다(④). 설계 초안에서는 Qwen3-Embedding-0.6b를 가정했지만, 측정 결과 교차언어 정확도가 낮아 **측정 결과에 따라 교체했다**. reranker는 질의당 약 4.5초가 더 들지만, 보고서 생성은 대화형이 아닌 배치 작업이므로 비용 대비 정확도 개선(MRR +0.073)이 크다고 판단해 채택했다. 시간이 부족할 때는 --no-rerank 옵션으로 끌 수 있다.

④ 구현 조건 재측정 (v2: 'max_seq_length' 2,048, 잘린 청크 0건)

v1 수치는 바꾸지 않고, 구현 설정으로 같은 42문항을 다시 측정했다. bge-m3-Qwen3는 2,048로, e5-MiniLM은 모델 한계 그대로 두었다. 새 순위에서 판정 이력이 없는 (질문, 청크) 쌍은 오답으로 처리했으므로 아래 값은 **하한값**이다. 보고

지표 top-5 안에 판정 이력이 없는 쌍은 dense bge-m3에서 0건, 채택 구성에서 5건이다
([outputs/eval/v2_maxlen2048/](#)).

모델 (평가 설정 길이)	Hit@1	Hit@5	MRR@10 (v1 → v2)	SW / HW MRR	min(SW, HW)
bge-m3 (2,048)	0.524	0.905	0.703 → 0.691	0.748 / 0.635	0.635
qwen3-embedding-0.6b (2,048)	0.500	0.810	0.631 → 0.631	0.770 / 0.493	0.493
multilingual-e5-large (512)	0.476	0.857	0.626 → 0.626	0.662 / 0.589	0.589
multilingual-minilm-l12 (128)	0.262	0.500	0.377 → 0.377	0.378 / 0.375	0.375

검색 구성	Hit@1	Hit@3	Hit@5	MRR@10
Dense (한국어 질의)	0.524	0.857	0.905	0.691
BM25 (한국어 질의)	0.548	0.810	0.881	0.697
BM25 (영어 재작성 질의)	0.643	0.905	0.929	0.770
2중 하이브리드 RRF: Dense(KO) + BM25(EN)	0.643	0.929	0.929	0.786
3중 하이브리드 RRF: Dense(KO) + Dense(EN) + BM25(EN)	0.643	0.881	0.952	0.778
2중 하이브리드 RRF + reranker (참고)	0.786	0.952	0.976	0.863
3중 하이브리드 RRF + reranker ← 채택(v1에서 고정)	0.786	0.952	0.976	0.863

해석: 임베딩 순위는 v1과 같다. bge-m3가 전체 Hit@1-3-5와 MRR에서 가장 높았고, min(SW, HW)도 bge-m3 0.635, Qwen3 0.493이다. bge-m3는 청크 전체를 인코딩하자 MRR이 0.703에서 0.691로 약간 낮아졌다(1문항 순위 변화 수준). 채택 구성(3중 RRF + reranker)은 v1과 같은 Hit@1 0.786 · Hit@5 0.976 · MRR 0.863이다. 2중 RRF + reranker도 같은 값이라, reranker를 거친 뒤에는 이 평가셋으로 두 하이브리드의 차이가 드러나지 않는다. 설계는 v1에서 고정한 3중 구성을 유지한다(결과를 보고 사후에 교체하지 않음).

평가의 한계와 향후 계획

- **과적합 가능성:** Qwen3 프롬프트 설정과 검색 구성을 최종 평가셋과 같은 42문항으로 골랐다. 42문항에서 1문항은 약 2.4%p이다. 예를 들어 bge-m3와 e5의 Hit@5 차이(0.905 vs 0.857)는 **2문항** 차이에 해당한다.
- **사후 정의 기준:** 진영 간 균형 지표는 측정 이후에 정의했다.
- **측정 조건:** v1 수치는 일부 청크가 잘린 조건(max_seq_length 1,024)에서 측정했다. 구현 조건(2,048)으로 다시 잰 v2 수치를 ④에 따로 적었다. v2는 판정 이력이 없는 쌍을 오답으로 처리했으므로 하한값이다.
- **향후 계획:** 문항을 확충한 뒤 개발셋(설정 선택)과 테스트셋(최종 보고)을 분리한다. 최종 성능은 테스트셋 수치로만 보고한다. 현재 42문항을 둘로 나누면 테스트셋이 약 20문항이 되어 1문항이 5%p를 차지하므로, 확충이 먼저 필요하다. 재실행 결과는 새 수치(v2 측정)로 구분해 표기하고 기존 수치는 보존한다. 최종 42문항의 정답 청크 **전수 사람 검수**를 권장 사항으로 둔다.

B.7 도구 정의 (외부 정보 검색·요약)

도구('@tool')	사용 에이전트	동작
<code>paper_retrieve(query, camp, tech, role, k)</code>	기술 조사, 시장, 도메인	B.5의 하이브리드 RAG. 메타데이터 필터를 쓰고, 근거 ID(P:<chunk_id>)-페이지-절을 반환
<code>web_search(query, days, stance)</code>	TRL(7~9 신호), 시장, 이해관계자, 도메인	Tavily 검색. 결과를 <code>data/web_cache/</code> 에 날짜 스탬프와 함께 캐시하고 <code>--offline</code> 이면 캐시만 사용. URL-게시일-발행 주체-벤더/제3자 라벨이 붙은 근거 ID(W:<hash>)를 반환
<code>summarize_sources(docs, focus)</code>	관점 에이전트 공통	근거 묶음을 초점별로 요약. 원 근거 ID를 보존 하는 구조화 출력이며, 근거 없는 문장은 만들지 않음

B.8 LLM 구성과 재현성

항목	설정
Generator	<code>gpt-4.1-mini</code> (키로 호출 가능한지 확인함, 비용 효율) · temperature 0 · seed 42
Judge	<code>gpt-4.1</code> (Generator보다 상위 모델 로 채점. 같은 GPT-4.1 계열이라 자기평가 편향 감소 효과는 제한적이며, 이 점을 한계에 적음)
캐시	LLM 응답(SQLite)과 웹 검색 결과를 repo에 커밋. <code>--offline</code> 이면 API 키 없이도 같은 보고서가 다시 생성됨
추적	LangSmith tracing, <code>outputs/logs/</code> 실행 로그, State의 <code>audit_log</code>

B.9 설계 단계 구현·검증 현황

설계서를 제출하는 시점에 이미 구현하고 측정한 항목이다. 모든 결과는 repo에서 재현할 수 있다.

항목	결과	재현 경로
코퍼스	6편, 136페이지(한도 200), 자동 검증 통과	<code>scripts/download_papers.py</code>
파싱·청킹	절 인식 청크 159개(최대 900 d100k 토큰), 참고문헌 제외	<code>rag/loader.py</code> , <code>rag/chunker.py</code>
평가셋	60 생성 → 57 QC → 50(포괄 7 제외) → 42(논문별 7, SW 21/HW 21). 42문항 중 복수 정답 29문항, 사람 검수 11문항(26%)	<code>eval/build_qa.py</code> , <code>eval/qa_review.md</code>
임베딩 선정	bge-m3(후보 4종 중 최고). v1(1,024): Hit@1 0.548 · Hit@5 0.905 · MRR 0.703 / v2(2,048): Hit@1 0.524 · MRR 0.691. 개발 지표	<code>eval/run_embedding_eval.py</code>
검색 구성	3중 하이브리드 + reranker. v1: Hit@1 0.786 · Hit@5 0.976 · MRR 0.863 / v2: Hit@1 0.786 · Hit@5 0.976 · MRR 0.863. 개발 지표(v2는 하한값)	<code>outputs/eval/</code> , <code>outputs/eval/v2_maxlen2048/</code>
측정 조건 점검	모델 고유 최대 입력과 평가 설정 비교, 잘린 청크 수(bge-m3 17, Qwen3 4)	<code>eval/check_truncation.py</code>
그래프	mermaid 설계를 로컬에서 렌더링(외부 업로드 없음)	<code>report/mermaid_render.py</code>

평가 과정에서 바로잡은 문제(Lessons):

- ① **관대한 정답 판정**: 초기 LLM 판정은 주제만 비슷해도 정답으로 보고, 다른 논문의 청크까지 정답 처리했다. 기준 답과 대조하는 엄격한 판정으로 바꾸고, 복합 질문을 단일 사실 질문으로 다시 생성했다.
- ② **재작성 질의의 누설**: 정답 청크를 본 LLM이 재작성한 질의가 BM25 점수를 부풀렸다(0.816). 질문만 보고 재작성하도록 바꿨다(0.770).
- ③ **프롬프트 민감도**: Qwen3는 직접 넣은 지시문보다 모델 카드 기본 프롬프트에서 더 좋았다. 후보마다 가장 좋은 설정으로 비교했다.
- ④ **토큰 기준 불일치(v1.1에서 발견)**: 청크 900토큰은 cl100k 기준이라, 임베딩 토큰나이저로는 최대 1,529 토큰이다. 평가 설정 1,024에서 일부 청크가 잘렸다는 사실을 공개하고, 구현 설정을 2,048로 올린다.

C. 평가 관점 및 기준: 도메인 선정 · 4가지 관점별 평가 대상 및 기준

C.1 도메인 선정과 문제 정의

도메인 후보	적합성 검토	채택
데이터센터·클라우드 장문맥 LLM 서버	두 기술 모두 KV cache가 급증하는 장문맥·대규모 배치 상황을 문제로 정의한다. 논문 실험 환경(GPU 서버, CXL 서버·vLLM)과 직접 일치하고, 산업 연관성(vLLM, CXL 표준)도 이 도메인에 집중되어 있다.	주 도메인
온디바이스·엣지 추론	자원·전력 제약이 커서 평가 기준이 달라진다. 두 기술 모두 직접 근거가 적어, 환경에 따른 평가 변화(H3)를 보는 대조군 으로만 쓴다.	대조군
멀티모달 서버	두 논문 모두 텍스트 기반 실험만 제공해 근거가 부족하다.	제외

주 도메인의 워크로드는 두 가지로 한정한다. **(W1) 장문맥 배치 추론**(문서 QA·요약, 32K~128K+)과 **(W2) 고동시성 다중 턴·에이전트 서버**(prefix KV 재사용)이다.

문제 정의 (선정 도메인 기반)

구분	내용
의사결정 상황	장문맥(수십~수백K 토큰)과 다중 턴·에이전트 워크로드가 늘면서, 서버 사업자는 GPU당 동시 세션 수와 최대 문맥 길이가 HBM 용량에 묶이는 문제 를 겪는다. GPU를 늘리는 것 말고, KV를 줄이는 SW 기술과 KV를 둘 공간을 늘리는 HW 기술이 대안으로 거론된다.
핵심 문제	같은 기술을 두고도 시장, 이해관계자, 도메인 관점마다 평가가 엇갈린다 . 예: 시장은 즉시 적용 가능성을 높이 보는데, 도메인 관점은 정확도 영향과 엔진 통합 수준을 문제 삼는다. 이 엇갈림을 근거 없이 한 방향으로 요약하면 의사결정이 왜곡된다.
분석 질문	Q1. 두 기술은 각 관점(TRL, 시장, 이해관계자, 도메인)에서 어떤 근거로 어떻게 인식되는가? Q2. 관점 간 일치·상충 지점은 어디이며, 그 원인은 무엇인가? Q3. 환경(데이터센터 vs 온디바이스)과 결합 사용 여부에 따라 평가 조건이 어떻게 달라지는가? (가설 H1~H4)
산출물 성격	기술 추천이나 우열 판정이 아니라, 관점별 인식 차이와 그 근거·조건을 추적할 수 있게 정리한 다관점 평가 보고서
1차 독자	LLM 인프라 기획·운영 담당자, 메모리 반도체 사업 기획 담당자

범위와 설계 원칙

- **범위 안:** SW 기술 1건(TurboQuant)과 HW 기술 1건(ITME)의 4관점 평가, 관점 간 상충 분석, 가설 H1~H4 검증, 자동 생성 보고서(PDF).
- **범위 밖:** 벤치마크 재현 실험, 기술 간 우열·추천 판정, 가격 추정 모델링.
- **중립성:** 두 기술에 같은 기준과 같은 질문 템플릿을 쓰고(대칭), 찬반 근거를 함께 검색하며, 우열 표현을 교정한다.
- **추적성:** 모든 주장에 근거 ID(논문 페이지 또는 URL)를 붙인다. 근거가 없는 주장은 Judge가 걸러낸다.
- **재현성:** 명령 한 줄로 실행된다. 웹 검색 결과와 LLM 응답을 캐시로 커밋하고, `--offline` 옵션으로 다시 재생할 수 있다.

C.2 공통 원칙

본 평가는 "어느 기술이 우수한가"를 판정하지 않는다. **같은 두 기술이 관점에 따라 어떻게 다르게 평가되는지**를 드러내는 것이 목표이다.

원칙	적용 방법
찬반 균형	관점별로 "장점·도입 근거"와 "한계·비판 근거"를 기술마다 각 2건 이상 확보한다. 한쪽만 확보되면 재검색(Loop)한다. 모든 근거에 <code>scope</code> (tech_specific 또는 category) 태그를 붙이고, category 근거(예: CXL 일반에 대한 비판)만으로는 할당량을 채울 수 없다 . 부족하면 거짓 균형을 만들지 않고 근거 부족 으로 둔다.
출처 다양성	한 출처군의 비중이 50%를 넘지 않게 한다. 분모 는 해당 관점·해당 기술의 최신 결과가 인용한 웹 근거 ID 수이다. URL은 등록 도메인(eTLD+1)으로 정규화하고, 같은 벤더의 사이트는 하나의 벤더 출처군 으로 묶는다(Google 계열: research.google, blog.google, deepmind.google 등 / SK hynix 계열: skhynix.com, news.skhynix.com 등). 모든 웹 근거에 벤더/제3자 라벨을 붙인다. 도메인이 달라도 같은 보도자료·블로그를 재인용한 기사 는 독립 근거가 아니므로, 근거마다 <code>origin_group</code> (원 출처 계열)을 붙인다. 50% 비중과 찬반 할당량은 독립 계열 수 기준으로 센다. 예: Google 블로그 → 기술 매체 기사 → 그 기사를 재인용한 기사는 1계열이다.
근거 추적성	모든 정량 주장과 정성 판단에 근거 ID(논문 페이지 또는 URL)를 1개 이상 연결한다. 근거가 없는 문장은 Judge가 unsupported 로 표시하고 보고서에서 뺀다.
대칭성	두 기술에 같은 평가 항목, 질문 템플릿, 검색 횟수 한도를 적용한다.
우열 표현 교정	"우수하다", "열등하다", "더 낫다" 같은 표현을 사전과 LLM으로 찾아내 "~ 조건에서 ~ 근거가 보고됨" 식의 조건부 서술로 바꾼다.

C.3 관점별 평가 대상 · 기준 · 증거 유형 · 출력 형식

관점 (담당)	평가 대상	평가 기준	증거 유형	출력 형식 (State)
① 기술 성숙도 · TRL (기술 조사 Agent)	각 기술의 원 논문, 공식 구현체, 제품화 신호	구현 검증 수준(시뮬레이션 / 실제 HW 실측 / 프로토타입), 재현 가능성(코드·데이터 공개), 서버 프레임워크 반영, 고객 샘플·제품 출시	논문 RAG(실험 조건), 웹(릴리스 발표)	<code>trl_result</code> : 기술별 TRL 단계, 근거표(출처·단계·공개 한계), 신뢰도

관점 (담당)	평가 대상	평가 기준	증거 유형	출력 형식 (State)
② 시장성 (시장 평가 Agent)	TurboQuant: 추론 서버 최적화 SW 세그먼트 / ITME: CXL 메모리 확장 HW 세그먼트	시장 규모·성장률, 상용화·채택 사례, 생태계 지원(vLLM·TensorRT·LLM·SGLang, CXL Consortium·OCP), 도입 비용 구조(재학습 불필요 vs HW 투자)	시장 리포트, 제품 발표, 프레임워크 문서, 논문 RAG(적용 조건)	<code>market_result</code> : 기준별 찬반 근거와 인식 점수
③ 이해관계자 (이해관계자 평가 Agent)	(a) 클라우드·데이터센터 사업자 (b) GPU·메모리 벤더(경쟁 진영 포함)(c) 개발자 커뮤니티(vLLM 등 OSS)(d) 투자·애널리스트·미디어	집단별 공식 입장·발언, 커뮤니티 반응(이슈·포럼), 대응 기술, 채택 장벽, 투자·표준화 참여	웹(기업 발표, 개발자 토론, 언론·분석)	<code>stakeholder_result</code> : 집단 × 입장(지지/우려/중립) 매트릭스와 근거
④ 도메인 적합성 (도메인 평가 Agent)	W1 장문맥 배치 추론, W2 고동시성 다중 턴 서버(+ 온디바이스 대조)	비용(TCO 구조), 지연(TTFT·TPOT), 처리량·동시성, 정확도 영향, 통합 난이도(엔진·드라이버·HW 변경), 온디바이스 제약(전력·메모리·HW 가용성)	논문 RAG(벤치마크 조건), 웹(실제 시스템 사례)	<code>domain_result</code> : 워크로드별 적합 조건·제약, 환경별 대조표

C.4 TRL 추정 규칙 (공개 정보 기반 추정)

TRL	단계 정의	KV cache 기술에서 인정하는 공개 증거 예시
1-3	원리 관찰 · 개념 정립 · 실험실 개념 검증	논문의 이론 분석, 시뮬레이션, 단일 GPU·오픈 모델 실험
4	실험실 환경 구성요소 검증	실제 HW(GPU·CXL 모듈)에서 구성요소를 통합해 실험
5	유사 환경 통합 검증	실제 서버 엔진(vLLM 등)과 실제 규모 워크로드에서 통합 검증
6	실제 환경 유사 조건 시스템 시연	프로토타입 시스템 시연, 공개 데모, 파트너 평가
7	실제 운용 환경 시제품 시연	고객사 파일럿·샘플 공급, 실서비스 시험 적용 발표
8	시스템 완성 · 양산 적합성	정식 제품 출시, 메인라인 프레임워크 정식 기능 탑재
9	실제 운용 · 상용 납품	상용 서비스 운영 사례, 양산 납품·실적 공시

공개 정보의 한계: TRL 1~3은 논문·학회로 대부분 공개되고, 7~9는 샘플·양산 발표로 일부 공개된다. **TRL 4~6은 수율·실측치가 영업 비밀이라 공개 정보 공백이 가장 크다.** KV cache 기술은 논문 발표와 실제 채택 사이에 시차도 있다. 따라서 TRL은 "공개 정보 기반 추정"임을 반드시 표기하고 단일 값이 아닌 범위(예: 4-5)와 신뢰도로 제시한다.

C.5 척도 · 가중치 · 상충 판정 규칙

- 세부 기준마다 **인식 점수 5점 척도**로 채점한다(1 부정적 인식 우세 · 3 혼재·중립 · 5 긍정적 인식 우세). 채점 근거 문장과 근거 ID를 반드시 함께 기록한다.
- 기준별 가중치는 **분석 전에 아래와 같이 고정**한다. 근거 수는 가중치에 쓰지 않고 신뢰도와 근거 충분성 판정에만 반영한다. 웹 자료가 많은 기술이 구조적으로 유리해지는 비대칭을 없애기 위해서다.

- 근거가 부족한 기준은 낮은 점수를 주지 않는다. **"판단 보류"**로 표시해 계산에서 빼고, 나머지 가중치를 재정규화한다. 빠진 가중치가 50%를 넘으면 그 관점 점수 자체를 "판단 보류"로 한다.

관점	기준 (고정 가중치)	점수 환산
시장성	시장 규모·성장 25 · 상용화·채택 사례 30 · 생태계 지원 30 · 도입 비용 구조 15	기준별 인식 점수의 고정 가중 평균
이해관계자	(a)클라우드·데이터센터 25 · (b) GPU·메모리 벤더 25 · (c) 개발자 커뮤니티 25 · (d) 투자·미디어 25	집단별 입장을 지지 5 / 혼재·중립 3 / 우려 1 로 환산한 고정 가중 평균
도메인 적합성	W1 50 · W2 50. 워크로드마다 비용·지연·처리량·정확도 영향·통합 난이도 각 20	워크로드별 기준 점수의 고정 가중 평균. 온디바이스 대조는 점수에서 제외(H3 전용)
TRL	점수화하지 않음. 단계 범위(예: 4-5)와 신뢰도로 제시	상충 판정에서 분리 . H1 전용 비교에만 사용

- **상충 판정**: 인식 점수를 가진 세 관점(시장·이해관계자·도메인) 사이에서, **기술 안에서만** 한다. 같은 기술의 두 관점 점수 차가 2.0 이상이면 "상충", 1.0 이상 2.0 미만이면 "부분 상충", 1.0 미만이면 "일치"이다. "판단 보류" 관점은 판정에서 뺀다. 기술 간 합계, 순위, 승패는 **산출하지 않는다**.
- **임계값의 성격**: 2.0-1.0 같은 상충 임계값과 H1 경계값은 기술의 절대적 평가 기준이 아니다. 관점 간 차이를 일관되게 분류하려고 **분석 전에 고정한 운영 규칙(heuristic)**이다. 보고서 5.5절에서는 임계값을 ± 0.5 바꿨을 때 분류가 달라지는 칸 수를 함께 보고한다(민감도 점검).
- **TRL을 분리한 이유**: 성숙도 척도와 인식 척도는 성격이 달라서, 섞으면 척도 차이만으로 H1이 기계적으로 "지지"될 수 있다. H1은 사전 규칙으로만 판정한다. **괴리**는 (TRL 상한 ≤ 4 이고 시장 ≥ 4.0) 또는 (TRL 하한 ≥ 7 이고 시장 ≤ 2.0)인 경우이고, 그 밖은 **일치**, 어느 한쪽이 판단 보류면 **판단 보류**이다.

관점 (보고서 출력 형식)	TurboQuant (SW)	ITME (HW)	비고
TRL	단계(범위) + 근거 요약 + 신뢰도	단계(범위) + 근거 요약 + 신뢰도	공개 정보 기반 추정, 상충 판정 제외 (H1 전용)
시장성	인식 점수 + 찬·반 근거 요약	인식 점수 + 찬·반 근거 요약	
이해관계자	인식 점수 + 집단별 입장 요약	인식 점수 + 집단별 입장 요약	
도메인 적합성	인식 점수 + 적합 조건 / 제약	인식 점수 + 적합 조건 / 제약	온디바이스 대조(점수 제외)
관점 간 상충	시장·이해관계자·도메인 간 일치 / 상충 + 가설 태그	시장·이해관계자·도메인 간 일치 / 상충 + 가설 태그	보고서 5장 재인용

C.6 검증 가설 (결론이 아닌 가설)

팀 가설과 초기 가설을 합쳐 4개로 정리했다. 보고서는 각 가설을 **지지 / 부분 지지 / 기각 / 판단 보류(근거 부족)**로 판정하고 근거 ID를 붙인다.

ID	가설	검증 방법
H1	기술 성숙도(TRL)와 시장 관점의 인식은 같은 방향을 가리키지 않을 수 있다. 논문·오픈소스 단계의 SW 기술이 시장 기대는 높고, 실측 검증된 HW 기술은 채택 신호가 제한적일 수 있다.	C.5의 H1 사전 규칙(TRL 범위 × 시장 인식 점수)으로 괴리·일치·판단 보류 판정
H2	이해관계자 반응은 기술 특성 자체보다 소속 생태계(빅테크 SW 생태계 vs 메모리·CXL 표준 생태계)의 영향을 더 많이 반영할 수 있다.	이해관계자 근거를 "기술 특성 언급"과 "생태계 전략 언급"으로 태깅해 비중 비교
H3	적용 환경(데이터센터 장문맥 서빙 vs 온디바이스)에 따라 두 기술의 적용 조건과 제약에 대한 평가가 달라질 수 있다.	도메인 평가의 환경별 대조표에서 평가 방향이 바뀌는 기준 확인
H4	두 접근은 경쟁보다 보완 관계일 수 있다(압축된 KV는 하위 계층으로 옮기기 쉬움). 즉시 적용성(SW)과 무손실 용량 확장(HW)은 서로 다른 채택 주체(기존 GPU 보유 기업 vs 신규 인프라 투자 기업)에게 호소할 수 있다.	결합 사용 근거 검색, 시장·이해관계자 근거를 채택 주체 유형별로 태깅

C.7 확증편향 방지 장치 요약

- 찬반 양면 검색:** 관점 에이전트는 기술마다 지지 질의와 반대 질의를 짝으로 실행한다.
- 출처 다양성 할당:** 원 출처 계열(재인용은 같은 계열) 기준으로 비중이 50% 이하가 되도록 하고, 벤더/제3자 라벨을 붙인다(ITME-TurboQuant 모두 1차 자료가 벤더 발표이므로 필수). 기술 고유(**tech_specific**) 근거만 찬반 할당량으로 인정한다.
- 인용 필수:** 근거가 없는 주장은 Judge가 삭제하거나 재작성을 요구한다.
- 대칭 처리:** 두 기술에 같은 템플릿, 같은 검색 한도, 같은 채점 기준을 적용한다.
- 상위 모델 Judge:** Generator(gpt-4.1-mini)보다 상위 모델인 gpt-4.1이 채점하고, **미달 관점만** 다시 실행한다. 같은 계열 모델이라 편향 감소 효과는 제한적이다(한계).
- 표현 점검:** 우열 어휘 사전으로 검사하고 LLM으로 교정한다(**final_check**).

D. 그래프 설계: State 설계 · Graph 흐름 설계

D.1 에이전트 설계

용어: Agent는 하나의 독립된 판단 책임 단위이고, Node는 LangGraph의 실행 단위이다. 하나의 Agent는 여러 Node로 구현될 수 있다. 예를 들어 기술 조사 Agent는 **selection_validator**, **tech_research**, **trl_assessor** 3개 노드로 구현된다. 전체는 에이전트 7개, 노드 18개이다.

Notion 가이드의 6개 에이전트를 그대로 두고, 중립성을 **코드로 보장하는** 품질 게이트인 Judge 1개를 추가했다. Judge는 새 사실을 만들지 않고 다른 에이전트의 산출물을 채점만 하므로 역할이 겹치지 않는다. 검색 품질 판정, 질의 재작성, PDF 변환은 LLM 판단 주체가 아닌 **보조 노드**로 두어 불필요한 에이전트를 만들지 않았다.

에이전트	구현 노드	역할	RAG	입력 State	출력 State	책임 경계 (하지 않는 일)
기술 조사	selection_validator tech_research trl_assessor	① 선정 검증 ② 원문에서 개요·실험 조건·한계 추출 ③ TRL 추정	0 (+Web : TRL 7-9 신호)	selected_techs retrieved_chunks	selection_validation tech_brief trl_result evidence	기술을 다시 고르지 않음, 시장·이해관계자 판단 안 함
시장 평가	market_evaluator	시장 규모, 채택 사례, 생태계 지원	0 + Web	tech_brief	market_result evidence	기술 원리 재조사 안 함
이해관계자 평가	stakeholder_evaluator	경쟁 진영·도입사·개발자·투자업계 시각	Web	tech_brief	stakeholder_result evidence	성능 수치 판단 안 함
도메인 평가	domain_evaluator	장문맥 서빙 적합 조건·제약, 온디바이스 대조	0 + Web	tech_brief	domain_result evidence	시장 전망 판단 안 함
평가 종합	synthesizer	관점 간 일치·상충 매트릭스, H1~H4 판정	X	4개 관점 결과, evidence	synthesis	새 검색 안 함, 기술 간 순위 산출 안 함
Judge	judge	관점별 근거성·종립성·다양성·완결성 채점, 결정적 검사, 미달 관점 지정	X	4개 관점 결과, synthesis, evidence	judge_scores failed_perspectives judge_feedback	내용을 직접 고치지 않음, 재실행 횟수 관리 안 함
보고서 생성	report_writer	목차대로 본문 작성, 인용·REFERENCE 정리	X	전체 결과, judge_scores, warnings	report_markdown references	근거 ID 없는 주장 추가 안 함

보조 노드: initialize, index_builder(로드·한도 확인·청킹·인덱싱), query_planner, hybrid_retriever, retrieval_grader, query_rewriter, retry_router(재실행 횟수 증가·Send 경로 생성), final_check, pdf_renderer

관점 에이전트 내부 루프: 각 관점 에이전트는 노드 안에서 자기교정 루프를 돈다. paper_retrieve/web_search로 검색하고, 관련성과 찬반 균형을 평가하고, 부족하면 재질의하는 과정을 **최대 2회** 반복한다. Judge가 재실행을 지시하면 judge_feedback[관점]을 추가 질의 조건으로 쓴다.

D.2 State 설계

병렬 노드가 같은 값을 동시에 덮어쓰지 않도록 **네 관점 결과를 각각 독립된 키에** 둔다. **reducer가 없는 키는 작성 노드가 하나뿐**이다. 여러 노드가 쓰는 키는 evidence, warnings, audit_log 세 개뿐이며, 병합 정책을 reducer로 명시한다. 관점 결과 키 4개는 누적하지 않고, 재실행 시 **최신 결과로 교체**한다.

State 키	타입	작성 노드	읽는 노드	설명 · 갱신 방식
run_id	str	initialize	전체	실행 추적 ID(LangSmith·로그 연결)

State 키	타입	작성 노드	읽는 노드	설명 · 갱신 방식
selected_techs	list[Technology]	initialize	selection_validator, query_planner	팀이 선정한 기술 2건(config / <code>--tech</code>), 진영, 선정 사유. 실행 중 변경 없음
selection_validation	SelectionValidation	selection_validator	report_writer	4개 기준 점수, 원문 근거, 약점, 통과 여부
document_manifest	list[DocumentMeta]	index_builder	report_writer	논문-페이지 수-해시-역할, 200p 한도 확인 결과
index_status	IndexStatus	index_builder	hybrid_retriever, report_writer	체크 수, 모델, 재사용 여부, 코퍼스 지문
queries	dict[str, list[Query]]	query_planner	hybrid_retriever	기술 × 개요 항목별 공통 질의
rewritten_queries	dict[str, list[Query]]	query_rewriter	hybrid_retriever	재작성 질의(있으면 <code>queries</code> 대신 사용, 재시도마다 교체)
retrieved_chunks	dict[str, list[Evidence]]	hybrid_retriever	retrieval_grader, tech_research	공통 RAG 검색 결과(체크-점수-페이지)
retrieval_grade	RetrievalGrade	retrieval_grader	라우터	관련성-범위-찬반 균형 판정과 누락 요소
retrieval_retry_count	int	query_rewriter	라우터	공통 검색 재시도 횟수(최대 2)
evidence	Annotated[list[Evidence], merge_by_id]	조사-평가 노드 (reducer)	synthesizer, judge, report_writer	논문(P:)-웹(W:)-근거 저장소. 병합 정책은 아래 참조
tech_brief	dict[str, TechBrief]	tech_research	4개 관점 노드, synthesizer	기술별 원리, 적용 범위, 실험 조건, 한계
trl_result	PerspectiveResult	trl_assessor	synthesizer, judge, report_writer	기술별 TRL 범위, 근거표, 공개 한계, 신뢰도
market_result	PerspectiveResult	market_evaluator	synthesizer, judge, report_writer	시장 규모-채택-생태계 찬반 근거와 점수
stakeholder_result	PerspectiveResult	stakeholder_evaluator	synthesizer, judge, report_writer	이해관계자 집단 × 입장 매트릭스
domain_result	PerspectiveResult	domain_evaluator	synthesizer, judge, report_writer	워크로드별 적합 조건-제약, 온디바이스 대조
synthesis	SynthesisResult	synthesizer	judge, report_writer	관점 × 기술 매트릭스, 상충 목록, H1~H4 판정
judge_scores	dict[str, JudgeScore]	judge	retry_router, report_writer	관점별 4항목 점수, 결정적 검사 결과, unsupported 주장 ID
failed_perspectives	list[str]	judge	retry_router	미달 관점(통과 시 빈 목록)
judge_feedback	dict[str, str]	judge	관점 노드(재실행 시)	관점별 보완 지시(추가 질의 조건)
perspective_retry_count	dict[str, int]	retry_router	retry_router, report_writer	관점별 재실행 횟수(각 최대 2). 한도에 이르면 <code>warnings</code> 에 "판정 불확실" 기록

State 키	타입	작성 노드	읽는 노드	설명 · 갱신 방식
report_markdown	str	report_writer	final_check, pdf_renderer	인용 번호가 연결된 보고서 원문
references	list[Reference]	report_writer	final_check, pdf_renderer	본문에 실제로 인용된 근거만 Notion 형식으로 정리
report_retry_count	int	final_check	라우터	보고서 형식 수정 횟수(최대 1)
report_pdf_path	str	pdf_renderer	app.py	생성된 PDF 경로
warnings	Annotated[list[str], add_unique]	여러 노드(reducer)	report_writer, final_check	근거 부족, 재시도 초과, 출처 편중 경고 → 보고서 한계점에 반영
audit_log	Annotated[list[AuditEvent], add]	전체(reducer)	final_check, 로그	노드, 시각, 질의, 사용 근거, 모델, 판정

‘evidence’ 병합 정책(merge_by_id): evidence_id는 (출처, 위치)의 해시이다. 논문은 doc_id, 페이지, 청크, 웹은 정규화 URL, 발췌 위치로 만든다. 같은 ID가 다시 들어오면 기존 항목을 유지하고 perspectives와 stances 필드만 합집합으로 합친다. claim이 서로 다르면 충돌로 판정해 warnings에 기록한다. 항목마다 attempt(관점별 실행 차수)를 두어, 재실행으로 대체된 이전 결과의 근거는 Judge 집계와 출처 50% 계산에 넣지 않는다.

주요 객체의 최소 구조는 다음과 같다.

```

Evidence      = {evidence_id, kind(paper|web), claim, summary, source_url|doc_id, title, publisher,
                 published_at, page, section, tech, perspectives{...}, stances{pro|con|neutral},
                 scope(tech_specific|category), source_class(vendor|third_party|academic),
                 source_group(eTLD+1 or vendor group), origin_group(원 출처 계열), attempt}
PerspectiveResult = {perspective, by_tech: {tech_id: {criteria: [{name, score_1to5, rationale, evidence_ids}],
                 summary, pro_ids, con_ids, limitations, confidence(high|mid|low)}}}
SynthesisResult  = {matrix: {tech_id: {perspective: {score, one_liner}}}, conflicts: [{tech_id, pair,
                 gap, explanation, hypothesis_tags}], hypotheses: {H1..H4: {verdict, rationale, evidence_ids}}}
JudgeScore       = {grounding, neutrality, source_diversity, completeness (1-5), passed,
                 unsupported_claim_ids, checks{max_origin_share, pro_origins, con_origins, lexicon_hits}}

```

D.3 Graph 흐름

흐름은 Human 선정(2안) → 초기화 → 선정 검증(기록만) → 공통 RAG(Loop) → 기술 조사 → 4개 관점 병렬 평가 (Fan-out) → 종합(Fan-in) → Judge → retry_router(Branch·선택적 재실행) → 보고서 생성·검수(Loop) → PDF 순서이다. 그림 2a는 전체 워크플로이고, 그림 2b는 품질 제어 루프를 펼친 상세도이다. 그림 2a의 Mermaid 소스는 아래와 같고(docs/graph_overview.mmd), 그림 2b의 소스는 docs/graph_quality.mmd에 있다. 구현이 끝나면 LangGraph의 draw_mermaid_png로 뽑은 실제 그래프(docs/graph.png)와 대조한다.

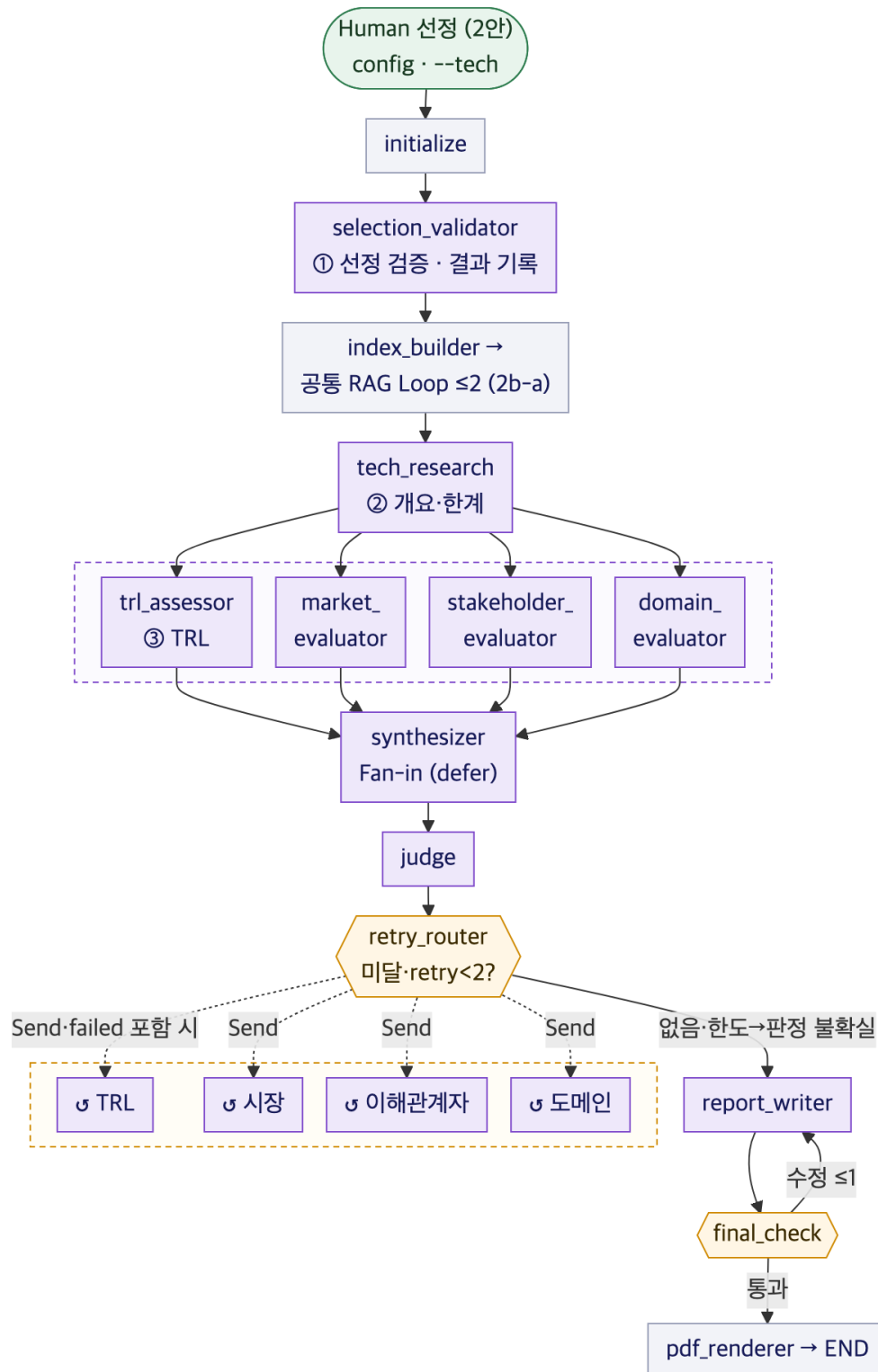


그림 2a. 전체 워크플로: 보라색은 에이전트 노드, 회색은 보조 노드, 노란색은 분기(Branch). 보라 점선 박스는 병렬 평가(Fan-out, 각 Agent 내부 Loop ≤2, 그림 2b-b)이다. 노란 점선 박스는 retry_router가 failed_perspectives에 포함된 관점에만 보내는 Send 재실행이며, 재실행 후 synthesizer로 돌아간다

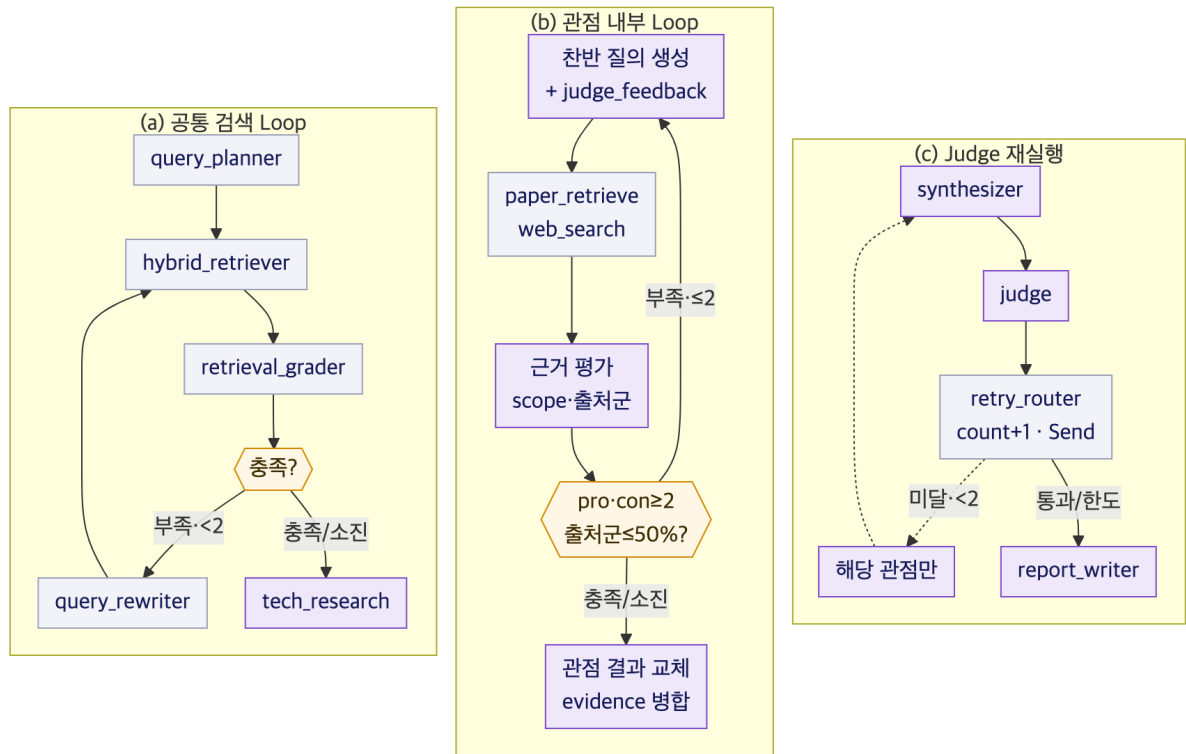


그림 2b. 품질 제어 상세: (a) 공통 검색 Loop(목적: 기술 개요와 공통 근거 확보), (b) 관점 에이전트 내부 Loop(목적: 해당 관점의 찬반 근거와 누락 기준 보완), (c) Judge 선택적 재실행 Branch

그림 2a Mermaid 소스 (Graph 흐름 설계)

```

flowchart TD
    H0(["Human 선정 (2안)<br/>config · --tech"]) --> INIT["initialize"]
    INIT --> VAL["selection_validator<br/>① 선정 검증 · 결과 기록"]
    VAL --> CR["index_builder -><br/>공통 RAG Loop ≤2 (2b-a)"]
    CR --> TR["tech_research<br/>② 개요·한계"]
    subgraph FAN[" "]
        direction LR
        TRL["trl_assessor<br/>③ TRL"]
        MK["market<br/>evaluator"]
        SH["stakeholder<br/>evaluator"]
        DM["domain<br/>evaluator"]
    end
    TR --> TRL & MK & SH & DM
    TRL & MK & SH & DM --> SY["synthesizer<br/>Fan-in (defer)"]
    SY --> JD["judge"]
    JD --> RR["retry_router<br/>미달·retry<2?>"]
    subgraph RERUN[" "]
        direction LR
        R1["< TRL"]
        R2["< 시장"]
        R3["< 이해관계자"]
        R4["< 도메인"]
    end
    RR --> R1 & R2 & R3 & R4
    R1 --> RR
    R2 --> RR
    R3 --> RR
    R4 --> RR
    RR --> RWR["report_writer"]
    RWR --> FC["final_check"]
    FC --> RWR
    FC --> PDF["pdf_renderer -> END"]
    classDef agent fill:#EEE8FA,stroke:#7F4ACB,color:#161A58;

```

```
classDef util fill:#F2F3F8,stroke:#9AA0B8,color:#161A58;
classDef human fill:#E8F4EC,stroke:#2E7D4F,color:#16381F;
classDef gate fill:#FFF6E5,stroke:#D08A00,color:#3A2A00;
class VAL,TR,TRL,MK,SH,DM,SY,JD,RWR,R1,R2,R3,R4 agent;
class INIT,CR,PDF util;
class H0 human;
class RR,FC gate;
style RERUN fill:#FFFDF6,stroke:#D08A00,stroke-dasharray:4 3,color:#3A2A00
style FAN fill:#FBFAFE,stroke:#7F4ACB,stroke-dasharray:4 3,color:#161A58
```

D.4 분기 · 합류 · 종료 규칙

- 선정 검증(분기 없음):** `selection_validator`는 검증 결과를 `selection_validation`에 쓰고, 미충족 항목은 `warnings`에 기록한 뒤 항상 `index_builder`로 진행한다. 2안(Human 선정)이므로 그래프 안에서 기술을 바꾸지 않는다(자동 재현성 확보). 검증 결과는 보고서 2.4절에 실린다.
- 공통 검색 루프:** `retrieval_grader`가 부족으로 판정하고 `retrieval_retry_count < 2`이면 `query_rewriter`로 가서 다시 `hybrid_retriever`로 돌아온다. 한도를 다 쓰면 경고를 남기고 `tech_research`로 진행한다.
- Fan-out:** `tech_research` 다음 4개 관점 노드를 같은 **superstep**에서 병렬로 실행한다. 노드마다 자기 결과 키에만 쓰므로 충돌이 없고, 공유 필드(`evidence`, `warnings`, `audit_log`)는 reducer로 병합한다.
- Fan-in:** `synthesizer`는 `defer=True`로 등록해 그 **superstep**에서 실행된 관점 노드가 모두 끝난 뒤 한 번만 실행된다. 선택적 재실행으로 일부 관점만 다시 돌 때도 멈추지 않는다.
- Judge 분기(Branch):** `judge`는 채점만 하고 `judge_scores`, `failed_perspectives`, `judge_feedback`를 쓴다. 이어지는 `retry_router`는 조건부 엣지 함수가 아니라 **노드**이다. LangGraph의 조건부 엣지 함수는 State를 갱신할 수 없기 때문이다. `retry_router`는 미달 관점 중 `perspective_retry_count < 2`인 관점의 횟수를 1 올리고, `Command(update=..., goto=[Send(관점 노드, {judge_feedback})...])`로 해당 관점만 다시 실행한다. 재실행 결과는 기존 결과를 교체하고 `synthesizer`와 `judge`를 다시 거친다. 한도에 이른 관점은 `warnings`에 "판정 불확실"로 기록하고, 재실행할 관점이 없으면 `report_writer`로 간다.
- 보고서 검수 루프:** `final_check`가 아래 항목을 확인한다. 실패하고 `report_retry_count < 10`이면 `report_writer`로 되돌린다. 순서는 먼저 검수하고 그다음 PDF 변환이다.
 - 핵심 주장마다 유효한 근거 ID가 있는가
 - REFERENCE가 실제 인용과 일치하는가
 - SUMMARY 분량이 ½페이지 이내인가
 - 우열 어휘가 남아 있는가
- 종료 보장:** 루프 4개 모두에 횟수 한도가 있다. 공통 검색 2, 관점 내부 2, 관점별 재실행 2, 보고서 수정 10이다. 최악의 경우에도 그래프가 반드시 끝난다.

D.5 Judge 통과 기준

관점마다 LLM 채점 4항목(5점 척도)과 **코드로 계산하는 결정적 검사**를 함께 적용한다. 판정 로직은 다음과 같다. 점수 합계(`sum ≥ 16`) 단독 조건은 쓰지 않는다.

```
passed = all(score >= 4 for score in [grounding, neutrality, source_diversity, completeness])
```

and max_origin_share <= 0.50 and pro_origins >= 2 and con_origins >= 2 and lexicon_hits == 0

항목	통과 기준	미달 시 조치
근거성 (LLM)	4점 이상. 핵심 주장마다 근거 ID가 있고 근거가 주장을 직접 뒷받침함	unsupported 주장 삭제 또는 재작성
중립성 (LLM)	4점 이상. 장점과 제약을 함께 제시하고, 근거 없는 우열 추천 표현이 없음	해당 관점 재작성
출처 다양성 (LLM + 코드)	LLM 4점 이상(1차 자료와 독립 자료 구분) 그리고 코드 검사에서 원 출처 계열 비중 ≤ 50%	해당 관점 재검색
완결성 (LLM)	4점 이상. 정의된 평가 기준과 두 기술을 모두 다룸	누락 기준 보완
찬반 균형 (코드)	기술별 tech_specific 근거의 독립 계열 수 $pro \geq 2, con \geq 2$	부족한 쪽 재검색. 한도 소진 시 근거 부족
우열 어휘 (코드)	사전 검사 0건	교정 후 재검사

D.6 구현 리스크와 대응

리스크	영향	대응
벤더 자료 편중 (ITME: SK hynix, TurboQuant: Google)	시장-이해관계자 평가가 발표 주체의 시각에 치우침	벤더/제3자 라벨, 도메인 50% 한도, 제3자 근거가 없으면 근거 부족 표기
ITME 공개 후 기간이 짧음(2026-06)	웹 반응-채택 근거가 부족함	기술 범주(CXL 메모리 확장)와 ITME 고유 근거를 구분해 표기, 판단 보류 허용
평가셋 규모(42문항)와 같은 셋으로 설정-구성까지 선택(개발/테스트 미분리)	과적합 가능성(1문항 = 2.4%p)	한계로 명시. 문항 확충 후 개발/테스트 분리, 정답 체크 전수 검수(B.6)
같은 계열 Judge(gpt-4.1-mini 생성, gpt-4.1 채점)	자기평가 편향 감소 효과 제한	코드 결정적 검사 병행, 한계 명시
평가 주체의 소속 편향(SK 교육과정 × SK hynix ITME)	ITME에 우호적인 해석 가능성	대칭 기준-벤더 출처군 상한-기술 고유 근거 할당량, 보고서 한계점에 명시
reranker 지연(질의당 약 4.5초)	실행 시간 증가	배치 작업이라 수용, --no-rerank 옵션 제공
선택적 재실행 시 합류 대기	그래프 정지	synthesizer defer=True , 관점별 재시도 한도
API 키 없는 채점 환경	재현 실패	LLM-웹 캐시 커밋, --offline 재생, 결과물 동봉

구현 계획: ① State-도구-보조 노트 → ② 기술 조사 Agent(검증-개요-TRL) → ③ 관점 에이전트 3종(내부 루프 포함) → ④ 종합-Judge-선택적 재실행 → ⑤ 보고서-검수-PDF(SKALA 양식) → ⑥ 새 환경에 clone해 **uv sync & uv run python app.py** 재현 검증 → ⑦ README(Notion 샘플 구조)와 발표 노트. 구현하다가 설계와 달라지는 부분은 docs/DECISIONS.md에 기록하고 이 문서를 갱신한다.

E. 평가 보고서 목차 (초안)

Notion 참고 목차의 순서(SUMMARY → 분석 배경 → 기술 선정 → 기술 개요 → 관점별 평가 → 시사점 → 한계점 → REFERENCE)를 그대로 따르고, 설계에서 필요한 항목(TRL, 상충 매트릭스, 가설 검증, 민감도)을 하위 절로 넣었다.

E.1 보고서 목차 (초안)

- **SUMMARY** (½페이지 이내, 개요가 아닌 핵심 발견)
- **1. 분석 배경**
 - 1.1 KV cache 병목: 연산 병목에서 메모리(HBM) 병목으로
 - 1.2 두 진영의 접근(SW: 데이터를 작게 / HW: 공간을 넓게)과 병용 가능성
 - 1.3 분석 도메인(데이터센터·클라우드 장문맥 서빙)과 문제 정의
 - 1.4 분석 질문(Q1~Q3)과 검증 가설(H1~H4)
- **2. 기술 선정**
 - 2.1 선정 방식(Human 기반, 2안)과 선정 기준
 - 2.2 후보 6개 기준표
 - 2.3 선정 결과와 사유(TurboQuant · ITME)
 - 2.4 에이전트 선정 검증 결과(약점 포함)
- **3. 기술 개요**
 - 3.1 TurboQuant: 핵심 접근, 보고된 성능, 적용 조건, 한계
 - 3.2 ITME: 핵심 접근, 보고된 성능, 적용 조건, 한계
 - 3.3 대칭 비교표
- **4. 관점별 평가**
 - 4.0 평가 기준 요약
 - 4.1 기술 성숙도(TRL): 공개 정보 기반 추정, 근거표
 - 4.2 시장성
 - 4.3 이해관계자
 - 4.4 도메인 적합성(W1·W2, 온디바이스 대조)
- **5. 시사점**
 - 5.1 관점 간 일치·상충 매트릭스
 - 5.2 주요 상충 지점 해설
 - 5.3 가설 H1~H4 판정
 - 5.4 조건별 시사점(추천 아님)
 - 5.5 임계값 민감도 점검(± 0.5)
- **6. 한계점**

- 6.1 공개 정보 기반 추정의 한계
- 6.2 확증편향 방지 조치와 실행 수치
- 6.3 분석 방법의 한계

• **REFERENCE** (본문에 인용된 자료만)

E.2 장별 구성과 원천 State

각 장은 State 키와 1:1로 대응하므로, 보고서 생성이 설계를 그대로 따르는지 추적할 수 있다.

장	구성	원천 State
SUMMARY	½페이지 이내. 관점 간 핵심 엇갈림 3~5개, H1~H4 판정 한 줄씩, "우열 판정 아님" 명시	<code>synthesis</code>
1. 분석 배경	1.1 KV cache 병목 · 1.2 두 진영의 접근과 병용 가능성 · 1.3 분석 도메인과 문제 정의 · 1.4 분석 질문과 가설	<code>selected_techs</code> , 설계 상수
2. 기술 선정	2.1 선정 방식(Human) · 2.2 후보 6개 기준표 · 2.3 선정 결과 · 2.4 에이전트 선정 검증 결과(약점 포함)	<code>selection_validation</code>
3. 기술 개요	3.1 TurboQuant · 3.2 ITME (접근, 보고된 성능, 적용 조건, 한계) · 3.3 대칭 비교표	<code>tech_brief</code>
4. 관점별 평가	4.0 평가 기준 · 4.1 TRL(공개 정보 기반 추정, 근거표) · 4.2 시장 · 4.3 이해관계자 · 4.4 도메인(온디바이스 대조). 모든 절에서 찬반 근거 병기	<code>trl_result</code> , <code>market_result</code> , <code>stakeholder_result</code> , <code>domain_result</code>
5. 시사점	5.1 관점 간 일치·상충 매트릭스 · 5.2 주요 상충 해설 · 5.3 가설 H1~H4 판정 · 5.4 조건별 시사점(추천 아님) · 5.5 임계값 민감도 (± 0.5)	<code>synthesis</code>
6. 한계점	6.1 공개 정보 기반 추정의 한계 · 6.2 확증편향 방지 조치와 수치 (찬반 비율, 출처 분포, Judge 점수·재실행 횟수) · 6.3 분석 방법의 한계(같은 계열 Judge, 평가 주체의 소속 편향 가능성, 판정 불확실 관점)	<code>judge_scores</code> , <code>warnings</code> , <code>perspective_retry_count</code>
REFERENCE	본문에 인용된 자료만. 논문·웹·특허 구분, Notion 표기 형식	<code>references</code>

E.3 작성 규칙

- **SUMMARY**는 개요가 아니라 핵심 발견만 담고 ½페이지를 넘지 않는다(`final_check`가 검사).
- 모든 주장에 근거 ID를 붙이고, **REFERENCE**에는 본문에 실제로 인용한 자료만 Notion 형식으로 적는다. 논문: 저자 (YYYY). 제목. 학회/학술지, 권(호), 페이지. · 웹: 기관(YYYY-MM-DD). 제목. 사이트, URL · 특허: 출원인 (YYYY-MM). 특허명, 번호, URL
- 기술 간 우열·추천을 쓰지 않는다. 우열 어휘는 0건이어야 한다.
- TRL은 "공개 정보 기반 추정"임을 표기하고 범위와 신뢰도로 제시한다.
- 파일명: RAG-Output_판교_9반_김정인+김지수+김진수+전진만+정원준.pdf (SKALA 양식)

참고자료

논문

- Zandieh, A., Daliri, M., Hadian, M., & Mirrokni, V.(2025). TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. arXiv, 2504.19874.
- Jang, H., Min, Y., Kim, S., Ahn, T., Kim, H., Joo, Y., Kim, H., & Kim, J.(2026). ITME: Inference Tiered Memory Expansion with Disaggregated CXL-Hybrid Memories. arXiv, 2606.12556.
- DeepSeek-AI(2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv, 2405.04434.
- Liu, Z., Yuan, J., Jin, H., Zhong, S. et al.(2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. ICML 2024 (arXiv, 2402.02750).
- Lee, W., Lee, J., Seo, J., & Sim, J.(2024). InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. OSDI 2024 (arXiv, 2406.19707).
- Kim, D., Lee, M., Kim, J. et al.(2025). Scalable Processing-Near-Memory for 1M-Token LLM Inference: CXL-Enabled KV-Cache Management Beyond GPU Limits. arXiv, 2511.00321.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z.(2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv, 2402.03216.
- Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F.(2024). Multilingual E5 Text Embeddings: A Technical Report. arXiv, 2402.05672.
- Zhang, Y. et al.(2025). Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. arXiv, 2506.05176.
- Reimers, N., & Gurevych, I.(2020). Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. EMNLP 2020, 4512-4525.
- Cormack, G. V., Clarke, C. L. A., & Büttcher, S.(2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009, 758-759.
- Grattafiori, A. et al.(2024). The Llama 3 Herd of Models. arXiv, 2407.21783.

기타(웹페이지)

- Google Research(2026-03-24). TurboQuant: Redefining AI efficiency with extreme compression. Google Research Blog, <https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/>
- 배기주(2026-08-12). KV cache 최적화 기술 평가. SKALA 과제 안내(Notion), <https://actually-war-1ea.notion.site/KV-cache-3ba7f4c866938099b7a8fdaa1831c07e>
- LangChain(2026-09-21 접속). LangGraph Graph API (Send, Command, defer, reducers). LangChain Docs, <https://docs.langchain.com/oss/python/langgraph/graph-api>